# 第十四届“恩智浦”杯全国大学生
# 智能汽车竞赛
# 室外光电创意组

# 技 术 报 告

参赛学校：上海第二工业大学

队伍名称：海豚队

参赛队员：黄子翀

赵一寰

彭小坤

带队教师：杨冠群

# 关于技术报告和研究论文使用授权的说明

本人完全了解第十四届“恩智浦”杯全国大学生智能汽车竞赛关保留、使用技术报告和研究论文的规定，即：参赛作品著作权归参赛者本人，比赛组委会和恩智浦公司可以在相关主页上收录并公开参赛作品的设计方案、技术报告以及参赛模型车的视频、图像资料，并将相关内容编纂收录在组委会出版论文集中。

参赛队员签名：

带队教师签名：

日　　期：2019/3/27

# 1.系统总体设计

## 1.1 系统概述

按照第十四届室外光电创意组比赛规则要求，我们在现有平台上通过某些机械调整与调用 ROS 系统软件包完成智能车的探路运行，实现车载激光雷达对赛道与其他车辆的检测和辨认，使用车载 IMU 确定自身方位，在 ROS 系统框架内利用电子地图完成按赛道运行并在运动中对赛道内的障碍物进行规避，通过 MCU 完成对于车辆的控制。

## 1.2 整车布局

在整车布局上基本沿用北京钢铁侠科技有限公司官方组装教程的布局。为了降低车辆重心，增加车辆的稳定性，我们将金属底板高度适度降低。移除塑料柱将激光雷达下移以最大化雷达效能，作为车上最重要的传感器，激光雷达的扫描效果直接影响了车辆的整体表现，好的雷达安装方式可以极大的减轻算法的复杂度并提升车辆运行时的稳定性。

总体来说，雷达的安装有以下两个目标：

1） 雷达视野不能被其他元器件遮挡

2） 雷达在车辆运行过程中保持平稳

关于激光雷达的安装位置，如果安装位置过低会导致大量其他电子元器件遮挡视野，减低激光雷达使用效能。若安装位置过高则会使得雷达无法扫描到其他车辆，同时障碍圆锥投影变小不利于躲避障碍。

为了实现上述两个目标，我们选择将原本安装在主结构板上方的工控机移动至主结构板下方，取消激光雷达的支撑柱，将激光雷达直接安装在主结构板上方，路由器拆除外壳后将电路板固定在主结构板下方，位于激光雷达正下方，自制驱动电路板安装在主结构板后部，设计时尽可能扁平化。以上设计力求在不遮挡激光雷达视野的前提下压低车高，

降低重心，增加车辆稳定性。

# 2.硬件系统

## 2.1 电路设计方案

电路设计部分使用 NXP 公司的 RT1052 作为主控芯片，该芯片拥有较高性能并且拥有丰富模块，同时拥有完善的底层库，易于开发。

传感器部分，为了简化主板设计，方便布线，激光雷达选择使用独立串口转 USB 模块连接主板。IMU 则直接使用连接线连接工控机。

主板接口部分，通过串口与工控机通讯，接受工控机的指令，通过 3PIN 插口与电调和舵机连接，发出控制指令。

供电部分为了简化设计，工控机直接由 3S 电池供电，后由 3S 电池引出的 11.6V 电通过 LM2940 稳压芯片降为 5V 直流电，供给舵机，现阶段为了快速完成样板，减少系统复杂程度，降低调试不确定因素，控制电路板使用 PC 的 USB 口供电，待初步调试结束后更换为线性稳压器供电。

详细设计见附录——电路结构图、电路原理图。

# 3.运行逻辑设计

## 3.1 基本运行逻辑设计

工控机在 ROS 系统框架下，通过激光雷达发回的地图数据与 IMU 发回的定位数据，结合电子地图制定运行策略，并发出任务指令控制单片机运行。单片机通过解析工控机发出的指令并在解析后将其转换为 PWM 信号实现对于舵机和电调的控制。

## 3.2 工控机部分软件设计

现阶段计划工控机通过串口通讯获得激光雷达的实时数据，通过 IMU 与电子地图实现对舵机的闭环控制。通过串口向单片机发送航向与

速度指令。为了规避障碍物以及防止与赛道边界碰撞，将电子地图与激光雷达获得的障碍物进行“膨胀”处理后提取出中间的可行赛道，之后将地图栅格化，利用寻路算法获取最佳路径，生成指令序列，将指令发送给单片机。

为了提高程序运行效率，提升控制速率，我们在全局代价地图的生成时采用整体计算，以便于 ROS 系统在进行全局规划时有一个宏观准确的路径。在局部代价地图生成时采用滑动窗口法，减少不必要的数据处理。同时适当降低代价地图更新速度，将更多计算机资源集中在近距离避障路径规划。

在启动阶段，为了防止过“硬”的启动使得 IMU 测得的加速度数据溢出，进而影响 ROS 系统对于车辆自身位置与运动状态的判断，我们在研究了车辆动力特性后，决定使用对数曲线作为车辆启动动力曲线，以使车辆在启动过程中尽可能的平稳。

## 3.3 单片机部分软件设计

单片机使用 DMA 接受工控机发来的串口数据，以 20HZ 作为控制频率对车辆进行控制。

通过使用 DMA 模块，可以在不打断单片机主控制流程的情况下完成串口数据的采集，实现单片机内数据输入与数据输出的分离，提高程序的执行效率与流畅性。

单片机通过 USB 转串口模块与工控机通讯，在单片机内完成数据的基本判断，使用 PWM 模块控制电调与舵机，实现车辆的运动。

## 3.4 ROS-MCU 协调部分软件设计

由于 ROS 系统的特性，其在数据获取，数据处理与指令发布上有无法避免的延迟。同时由于计算平台的性能限制，在处理复杂路况时 ROS 系统的控制有超时可能。介于以上两点，单片机在接受 ROS 系统下发指令时采用异步方式，最大限度的避免由于 ROS 系统延迟导致整辆智能车出现程序卡死的问题。

## 4.调试小结

近年来机器人领域不断发展，ROS 在该领域独占鳌头，其中著名的百度无人驾驶汽车所采用的 Apollo 系统就是基于 ROS 操作系统优化得来的，足以证明 ROS 操作系统的稳定性和广泛的应用价值。

ROS 底层基本由高效率的 C++编写，并且集成了易用的 Python 接口，实现了库可以不依赖于 ROS。由于其分布式框架的设计，节点可以分布于不同的主机，可以互为服务器或客户端，方便负载均衡。ROS 还提供一组实用工具和软件库，用于维护、构建、编写和执行可用于多个计算平台的软件代码。

得益于 ROS 操作系统的易用性和北京钢铁侠公司为我们提供了大量详实的文字资料和网上视频，我们很快便入门了 ROS 系统，较快的熟悉了钢铁侠公司为我们提供的整套平台。

由于 ROS 属于非实时操作系统，其 topic 通信机制使得大量的系统资源都浪费在通讯上，即使使用了桌面级的处理器在面对复杂的赛道环境时我们的平台也会出现一些运算超时的情况，考虑到正式比赛还会有其他车模作为干扰信号，比起训练赛道而言复杂度更胜一层楼，我们降低了部分非关键运算的执行频率，并且降低了 ROS 系统在处理代价地图时的分辨率并降低了局部地图的计算范围，以求将运算能力分配在少数关键步骤上，防止 ROS 系统对于车辆的控制出现严重偏差，提高平台的资源利用效率。

## 5.运行效果

起初我们直接使用官方提供的 move base 框架顺利的实现了小车自主导航的功能，但是运行速度仅有 0.5m/s 左右，距离比赛要求相差甚远。主要问题在于小车启动时 IMU 中的加速度计会受到较大震动的干扰致使 IMU 与地图定位不准确，于是我们对启动和制动都做了平滑处理，有效抑制了惯性导航的漂移，在此阶段我们的车模运行速度大约为 1m/s。

我们还陆续发现了规划路径被限幅、局部规划路径与全局规划路径周期冲突等问题，在查阅相关资料后我们解决了部分问题。由于整车模质量较重、重心较高、转弯时车模侧倾严重致使雷达外侧已经无法扫描到赛道围挡，雷达内侧直接扫描到地面，因此在此阶段我们只能调试到1.3m/s 左右的速度。

我们利用原装的钣金对车模的机械部分进行了可恢复性的改装，将车模整体重量减轻并将车模质量重心下移，降低了雷达的高度（改装后见附录中的车模三视图）。有效抑制了车身转向不足和雷达扫射面偏移的情况。

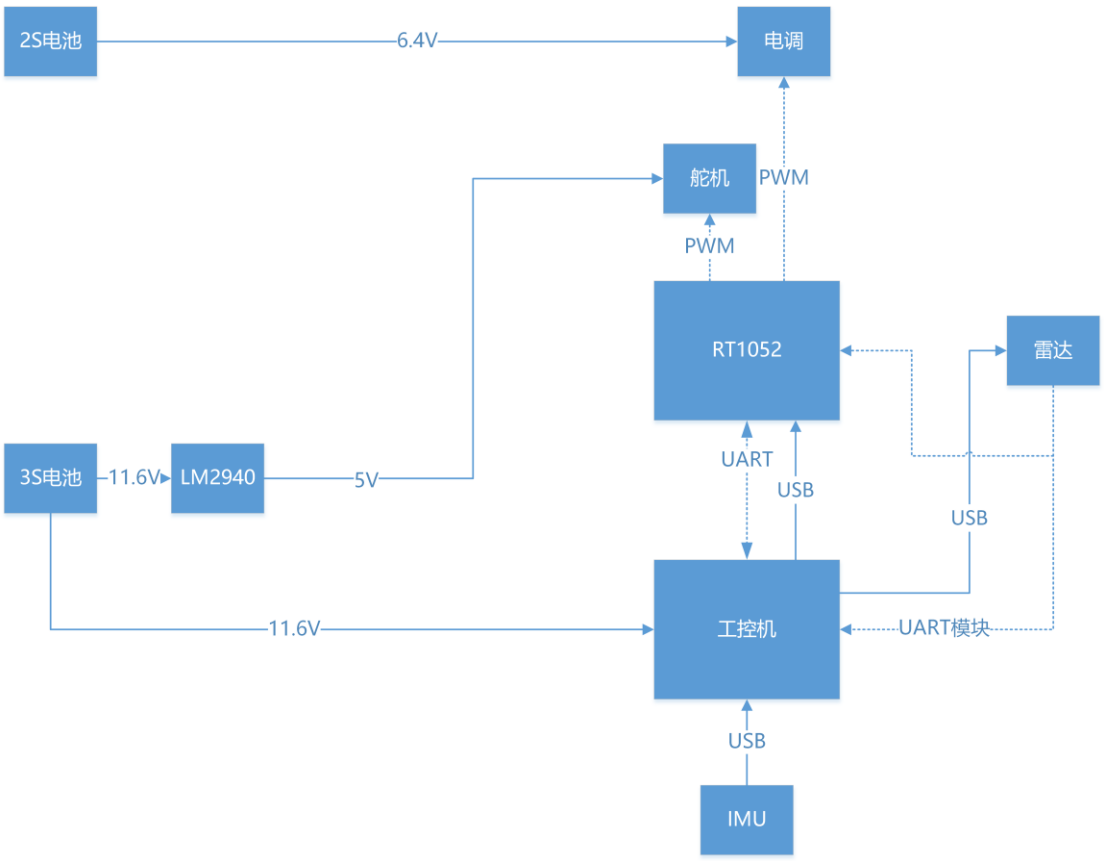
最终我们的车模可以以2.0m/s的平均速度完成总长度18m且设置有16个路障的赛道，相比于比赛要求我们将障碍物的密度提高了数倍（见附录视频），在减少至 8 个路障的情况下我们的最高速度可以提升至2.5m/s 左右。我们认为比赛时会有更厉害的选手，并且在多辆车模运行时难度会比静态路障更大，所以我们此后一直在高密度的障碍赛道中训练。

## 6.下一步计划

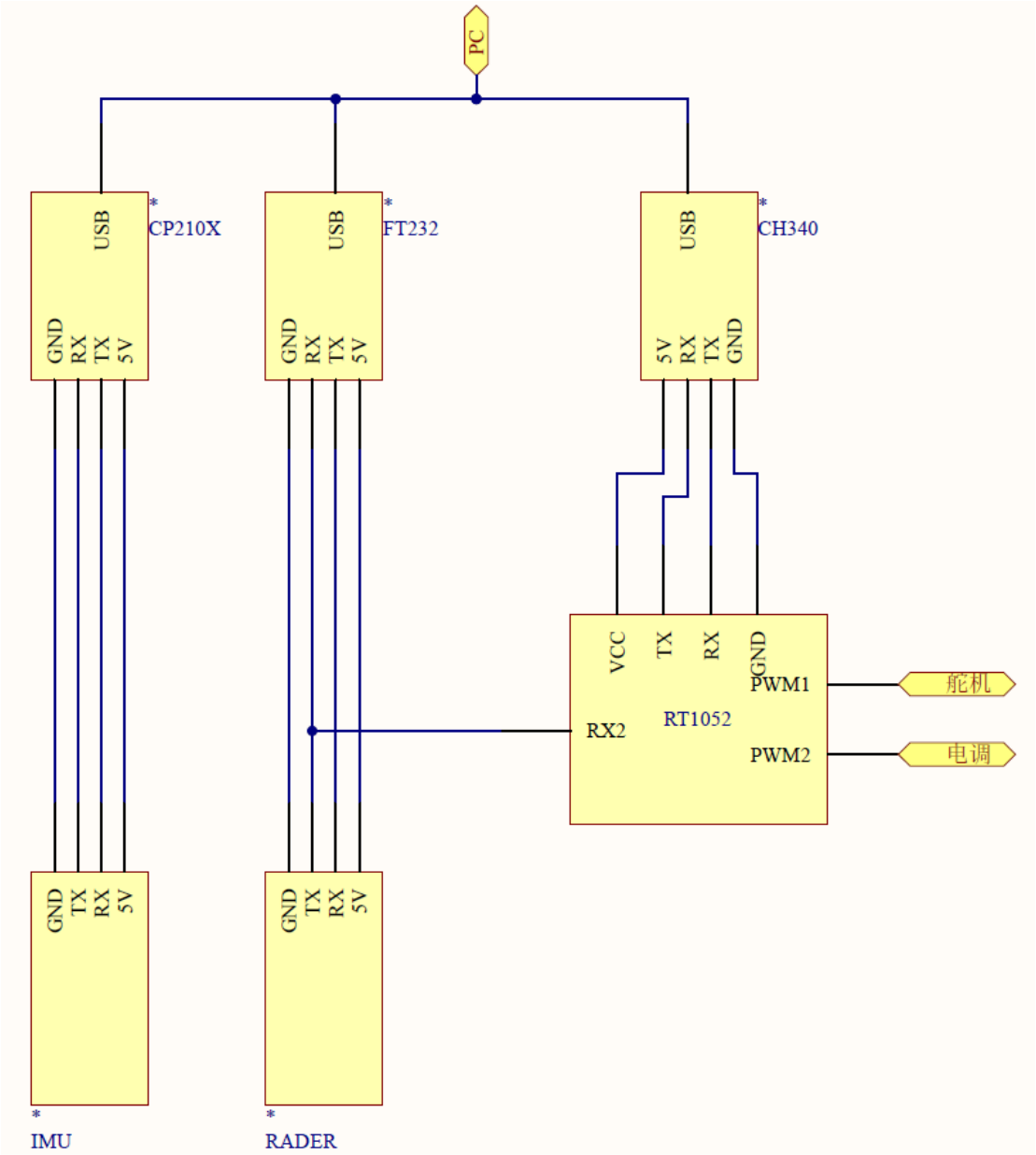
至此我们车模的完成度还远远达不到最终决赛的要求，其他多名对手移动的车模才是我们最大的难关，我们必须设法增加移动的干扰目标，甚至增加视觉传感器以更快更准确的捕捉到对方的位姿，以此做出更加准确的路径规划。我们还会想办法优化 ROS 系统中的通讯部分，降低通讯延迟获得更多的处理时间，为了提高定位速度我们也会尝试使用 ROS 中可集成的其他定位融合算法，路径规划也要做的更加保守一点。目前的局部规划路径还存在着一些问题没有解决，我们也会利用剩余的时间努力攻克这一系列问题，争取不浪费组委会以及钢铁侠公司为我们免费提供的车模，我们一定好好把握机会，争取做到最好。

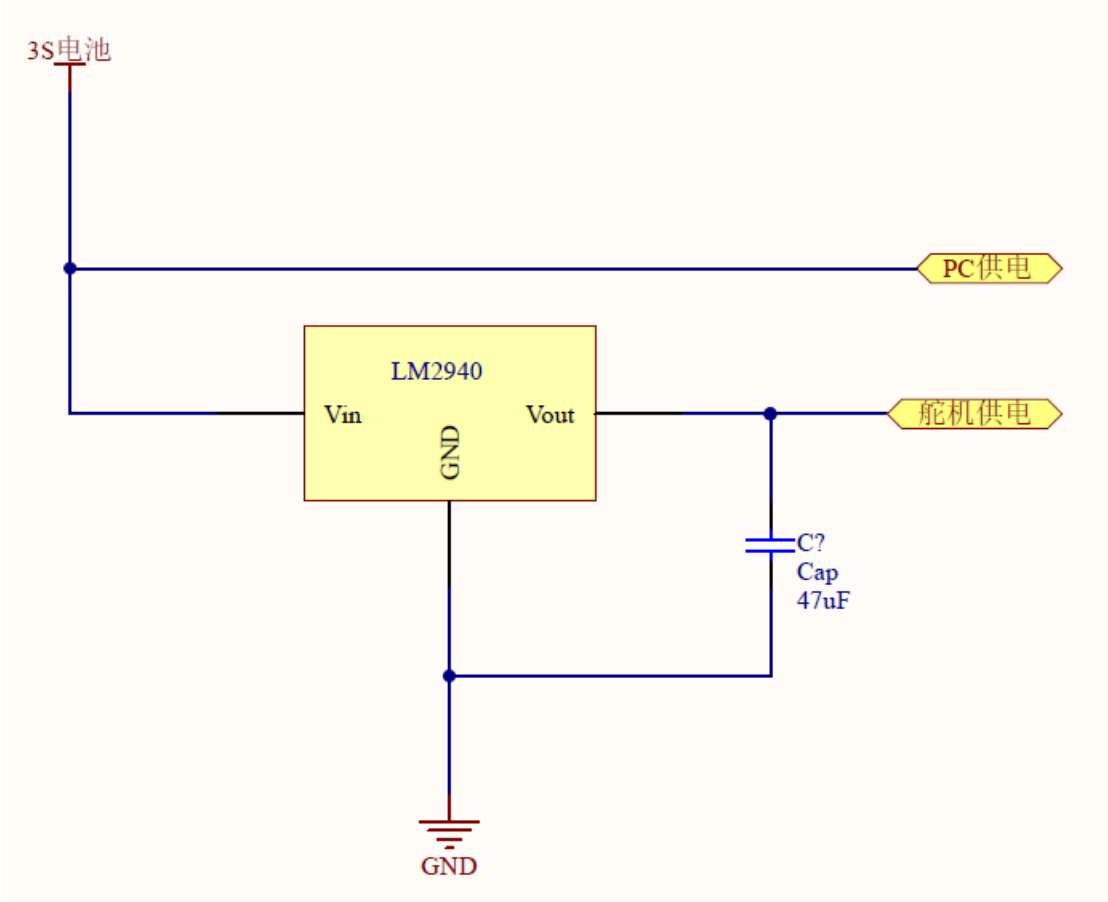
附录：

电路结构图：




电路原理图：


PC
USB
GND
RX
TX
5V
CP210X
USB
GND
RX
TX
5V
FT232
USB
5V
RX
TX
GND
CH340
VCC
TX
RX
GND
PWM1
RT1052
RX2
PWM2
舵机
电调
GND
TX
RX
5V
IMU
GND
TX
RX
5V
RADER




车辆三视图：

